

Cvičné úlohy ke státnicím

Úroveň 3 (doplněk k úrovním 1+2; dohromady ~97-98 %)

Informatika - Umělá inteligence, zaměření Strojové učení (UI-SU) | MFF UK

Aditivní díl: obsahuje POUZE rozšíření úrovně 3 a zkoušky nanečisto. Nejdřív zvládní úrovně 1 a 2.

Co je tohle a poctivý strop

Tři sešity jsou **aditivní**: úroveň 1 ($\approx 92\%$) + úroveň 2 ($\rightarrow \approx 95\%$) + tento díl = **$\approx 97,5\%$** . Tady jsou jen *nové* úlohy úrovně 3 a zkoušky nanečisto. Díl cílí na *zbytkové* riziko: vzácné, ale legální položky požadavku a chyby z formátu/času. Přidává:

- **Zbylý ocas požadavku** (aby žádný draw nebyl nepokrytý): algebraické struktury (grupy, tělesa), vektorové prostory a lineární zobrazení, posloupnosti a řady, uspořádání (Dilworth), konkrétní rozdělení + nerovnosti, odhady (MLE, intervaly); běh/JIT/linkování, správa zdroje a výjimky, generika, garbage collection; multiclass softmax + křížová validace, POMDP, predikátová rezoluce s unifikací, plánování regresí.
- **Dvě zkoušky nanečisto** (8 otázek se správnou strukturou) a **tvrdý specializační drill** - trénink formátu, časování a strategie parciálních bodů, bodováno podle reálných podlah.

Poctivý strop: z 9 zkoušek dat a velkého oficiálního okruhu je realistická hranice pilného studenta **$\sim 97,5\%$** (98% optimisticky); $> 99\%$ *nelze* z konečné sady poctivě tvrdit. Zbytek je nepředvídatelné frázování a lidská chyba - proti té pomáhají zkoušky nanečisto.

Obsah

1	Společná matematika (rozšíření)	1
2	Společná informatika (rozšíření)	5
3	Specializace: Strojové učení (rozšíření)	8
4	Specializace: Základy UI (rozšíření)	9
5	Zkoušky nanečisto a drill	11

1 Společná matematika (rozšíření)

Úloha 1. **společná matematika** [úroveň 3] **Algebraické struktury: grupy, podgrupy, tělesa, konečná tělesa**

- (a) **1 b** Definujte grupu a podgrupu. Co je permutace a grupa permutací S_n ?
- (b) **2 b** Definujte těleso. Uveďte příklady (i konečné) a vysvětlete charakteristiku tělesa.
- (c) **3 b** Zformulujte Lagrangeovu větu a uveďte, kdy je $\mathbb{Z}_n$ těleso.

(a) Grupa $(G, \cdot)$ je množina s binární operací, která je asociativní ($a(bc) = (ab)c$), má *neutrální prvek* e ($ae = ea = a$) a ke každému a *inverzní* a^{-1} ($aa^{-1} = e$). Je-li navíc komutativní, je *abelovská*. Podgrupa $H \leq G$ je podmnožina, která je sama grupou vůči téže operaci (uzavřená na operaci i inverzy, obsahuje e). Permutace množiny je bijekce na sebe; všechny permutace $\{1, \dots, n\}$ se skládáním tvoří grupu S_n řádu $n!$ (obecně nekomutativní).

(b) *Těleso* $(T, +, \cdot)$ je množina se dvěma operacemi, kde $(T, +)$ je abelovská grupa (s neutrálním 0), $(T \setminus \{0\}, \cdot)$ je abelovská grupa (s neutrálním 1) a platí distributivita. Příklady: $\mathbb{Q}, \mathbb{R}, \mathbb{C}$ (nekonečná) a $\mathbb{Z}_p$ pro prvočíslo p (konečné). *Charakteristika* je nejmenší $k > 0$ s $\underbrace{1 + \dots + 1}_k = 0$

(nebo 0, pokud takové neexistuje); u konečného tělesa je to vždy prvočíslo p a těleso má p^m prvků.

(c) *Lagrangeova věta*: v konečné grupě řád každé podgrupy dělí řád grupy ($|H| \mid |G|$); důsledek: řád každého prvku dělí $|G|$. $\mathbb{Z}_n$ (zbytky modulo n) je *těleso právě tehdy, když je n prvočíslo* - jen tehdy má každý nenulový prvek multiplikativní inverz (jinak existují dělitelé nuly).

Grupa = asociativní operace + neutrální + inverzy. Těleso = $(T, +)$ a $(T \setminus \{0\}, \cdot)$ abelovské grupy + distributivita. $\mathbb{Z}_p$ těleso $\iff p$ prvočíslo. Lagrange: $|H| \mid |G|$.

Kalibrace: Algebraické struktury (grupy, tělesa, konečná tělesa) jsou v požadavku, ale v moderních zkouškách vzácné - úroveň 3 je pokrývá, aby žádný draw společné matematiky nebyl nepokrytý.

Pozor (častá chyba): Těleso vyžaduje inverzy i pro násobení (kromě 0) - proto $\mathbb{Z}_4$ není těleso (2 nemá inverz). Charakteristika konečného tělesa je vždy prvočíslo.

Úloha 2. **společná matematika** [úroveň 3] **Vektorové prostory a lineární zobrazení**

- (a) **1 b** Definujte vektorový prostor, lineární nezávislost, bázi, dimenzi a souřadnice vektoru.
- (b) **2 b** Zformulujte Steinitzovu větu o výměně. Definujte lineární zobrazení, jeho obraz a jádro.
- (c) **3 b** Zformulujte větu o dimenzi (rank-nullity) a vysvětlete izomorfismus vektorových prostorů.

(a) *Vektorový prostor* nad tělesem T je množina V se sčítáním (abelovská grupa) a násobením skalárem $T \times V \rightarrow V$ splňujícím distributivitu, asociativitu a $1 \cdot v = v$. Vektory $v_1, \dots, v_k$ jsou *lineárně nezávislé*, pokud $\sum c_i v_i = 0$ implikuje všechna $c_i = 0$. *Báze* = lineárně nezávislý systém generátorů (každý vektor je jejich jednoznačnou lineární kombinací); *dimenze* = počet prvků báze. *Souřadnice* vektoru v bázi jsou koeficienty této kombinace.

(b) *Steinitzova věta o výměně*: je-li $\{v_1, \dots, v_k\}$ lineárně nezávislá a $\{w_1, \dots, w_m\}$ systém generátorů, pak $k \leq m$ a lze k generátorů vyměnit za v_i tak, že vznikne opět systém generátorů (důsledek: všechny báze mají stejnou velikost). *Lineární zobrazení* $f : V \rightarrow W$ splňuje $f(au + bv) = af(u) + bf(v)$. *Obraz* $\text{Im } f = \{f(v) : v \in V\}$ (podprostor W), *jádro* $\text{Ker } f = \{v : f(v) = 0\}$ (podprostor V).

(c) *Věta o dimenzi (rank-nullity)*: pro lineární $f : V \rightarrow W$ s konečnou dimenzí V platí $\dim \text{Im } f + \dim \text{Ker } f = \dim V$ (hodnost + defekt). *Izomorfismus* je bijektivní lineární zobrazení; dva prostory nad týmž tělesem jsou izomorfní právě tehdy, mají-li stejnou (konečnou) dimenzi - každý n -rozměrný prostor je izomorfní T^n (přes souřadnice v bázi).

Báze = nezávislý systém generátorů; dimenze = její velikost (všechny báze stejně velké, Steinitz). f lineární: $\text{Im } f \leq W$, $\text{Ker } f \leq V$, $\dim \text{Im } f + \dim \text{Ker } f = \dim V$. Izomorfní $\iff$ stejná dimenze.

Kalibrace: Vektorové prostory a lineární zobrazení (báze, dimenze, jádro/obraz, rank-nullity) jsou jádrem lineární algebry; úroveň 3 doplňuje abstraktní část vedle výpočetních úloh z úrovně 1-2.

Pozor (častá chyba): Báze není „libovolná generující množina“ - musí být i nezávislá. Rank-nullity

sčítá dimenze obrazu a jádra do dimenze *definičního oboru*, ne cíle.

Úloha 3. **společná matematika** [úroveň 3] **Posloupnosti, limity a řady**

- (a) **1 b** Definujte limitu posloupnosti a konvergenci. Uveďte aritmetiku limit a větu o dvou polícajtech.
- (b) **2 b** Spočtete $\lim_{n \rightarrow \infty} \frac{2n^2 + 1}{n^2 + n}$ a $\lim_{n \rightarrow \infty} \sqrt[n]{n}$.
- (c) **3 b** Definujte součet řady a uveďte kritéria konvergence (srovnávací, podílové, odmocninové). Proč harmonická řada diverguje a geometrická (pro $|q| < 1$) konverguje?

(a) $\lim_{n \rightarrow \infty} a_n = L$, pokud $\forall \varepsilon > 0 \exists n_0 \forall n \geq n_0 : |a_n - L| < \varepsilon$ (posloupnost je pak *konvergentní*). *Aritmetika limit*: limita součtu/součinu/podílu je součet/součin/podíl limit (u podílu při nenulové limitě jmenovatele). *Věta o dvou polícajtech*: je-li $a_n \leq b_n \leq c_n$ a $\lim a_n = \lim c_n = L$, pak $\lim b_n = L$.

(b) $\lim_{n \rightarrow \infty} \frac{2n^2 + 1}{n^2 + n} = \lim_{n \rightarrow \infty} \frac{2 + 1/n^2}{1 + 1/n} = \frac{2}{1} = 2$ (vydělením n^2). $\lim_{n \rightarrow \infty} \sqrt[n]{n} = 1$ (např. $\sqrt[n]{n} = e^{\frac{\ln n}{n}}$ a $\frac{\ln n}{n} \rightarrow 0$).

(c) *Součet řady* $\sum a_n$ je limita částečných součtu $s_N = \sum_{n=1}^N a_n$ (pokud existuje). *Srovnávací*: $0 \leq a_n \leq b_n$ a $\sum b_n$ konverguje $\Rightarrow \sum a_n$ konverguje. *Podílové (d'Alembert)*: $q = \lim |a_{n+1}/a_n|$; $q < 1 \Rightarrow$ konverguje (absolutně), $q > 1 \Rightarrow$ diverguje, $q = 1$ nerozhoduje. *Odmocninové (Cauchy)*: stejně podle $q = \lim \sqrt[n]{|a_n|}$ (< 1 konverguje, > 1 diverguje, $= 1$ nerozhoduje). *Harmonická* $\sum \frac{1}{n}$ diverguje (částečné součty rostou jako $\ln N$); *geometrická* $\sum q^n$ pro $|q| < 1$ konverguje k $\frac{1}{1-q}$ (částečný součet $\frac{1-q^{N+1}}{1-q}$).

Limita posloupnosti: $\forall \varepsilon \exists n_0 : |a_n - L| < \varepsilon$. Racionální podíl polynomu: vyděl nejvyšší mocninou. Řada konverguje dle podílového/odmocninového/srovnávacího kritéria; harmonická diverguje, geometrická ($|q| < 1$) $\rightarrow \frac{1}{1-q}$.

Kalibrace: Posloupnosti a řady jsou v požadavku analýzy a opakovaně se zkouší (geometrická řada byla samostatná otázka). Definice limity + jeden výpočet = 2 body.

Pozor (častá chyba): Podílové kritérium s $q = 1$ nerozhoduje. Harmonická řada diverguje, ač $a_n \rightarrow 0$ - samotné $a_n \rightarrow 0$ konvergenci nezaručuje.

Úloha 4. **společná matematika** [úroveň 3] **Částečná uspořádání: výška, šířka a věta o dlouhém a širokém**

- (a) **1 b** Definujte částečné uspořádání, řetězec a antiřetězec a minimální/maximální prvek.
- (b) **2 b** Definujte výšku a šířku částečně uspořádané množiny.
- (c) **3 b** Zformulujte větu o dlouhém a širokém (Dilworth/Mirsky) a ilustруйте na příkladu (dělitelnost na $\{1, 2, 3, 4, 6, 12\}$).

(a) *Částečné uspořádání* $\leq$ je reflexivní, antisymetrická a tranzitivní relace. *Řetězec* = podmnožina po dvou *porovnatelných* prvků; *antiřetězec* = podmnožina po dvou *neporovnatelných* prvků. *Minimální* prvek nemá nic ostře pod sebou (*nejmenší* je pod vším); *maximální* nemá nic nad sebou.

(b) *Výška* = velikost nejdelšího řetězce. *Šířka* = velikost největšího antiřetězce.

(c) *Věta o dlouhém a širokém*: pro každou konečnou uspořádanou množinu platí výška $\cdot$ šířka $\geq |X|$ (nelze mít zároveň jen krátké řetězce a jen úzké antiřetězce). Plyne z *Mirskyho věty*

(množinu lze pokrýt „výška“ antiřetězci, každý velikosti $\leq$ šířka) - to je silnější tvrzení; duálně *Dilworthova věta*: minimální počet řetězcu na pokrytí množiny = šířka. Příklad (dělitelnost na $\{1, 2, 3, 4, 6, 12\}$): nejdelší řetězec $1 \mid 2 \mid 4 \mid 12$ má 4 prvky (*výška* = 4), největší antiřetězec $\{4, 6\}$ má 2 prvky (*šířka* = 2); $4 \cdot 2 = 8 \geq 6$.

Řetězec = porovnatelné prvky, antiřetězec = neporovnatelné. Výška = nejdelší řetězec, šířka = největší antiřetězec. Dilworth: min. počet řetězcu na pokrytí = šířka; Mirsky: min. počet antiřetězcu = výška.

Kalibrace: Výška/šířka a věta o dlouhém a širokém jsou explicitně v požadavku diskrétní matematiky, ale vzácné; úroveň 3 je pokrývá pro úplnost.

Pozor (častá chyba): Dilworth páruje řetězce se šířkou (antiřetězcem), Mirsky antiřetězce s výškou - nezaměň směr. Minimální prvek nemusí být nejmenší.

Úloha 5. společná matematika [úroveň 3] Konkrétní rozdělení a nerovnosti

- (a) **1 b** Definujte binomické, geometrické, Poissonovo, normální a exponenciální rozdělení a uveďte jejich střední hodnotu.
- (b) **2 b** Zformulujte Markovovu a Čebyševovu nerovnost a jednu použijte.
- (c) **3 b** Zformulujte zákon velkých čísel a vysvětlete vztah binomické $\rightarrow$ Poissonovo/normální.

(a) *Binomické* $\text{Bi}(n, p)$: počet úspěchu v n nezávislých pokusech, $P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$, $\mathbb{E}X = np$. *Geometrické*: počet pokusu do prvního úspěchu, $P(X = k) = (1-p)^{k-1} p$, $\mathbb{E}X = 1/p$. *Poissonovo* $\text{Po}(\lambda)$: $P(X = k) = e^{-\lambda} \lambda^k / k!$, $\mathbb{E}X = \lambda$. *Normální* $\mathcal{N}(\mu, \sigma^2)$: hustota $\frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}$, $\mathbb{E}X = \mu$. *Exponenciální* $\text{Exp}(\lambda)$: hustota $\lambda e^{-\lambda x}$ pro $x \geq 0$, $\mathbb{E}X = 1/\lambda$.

(b) *Markovova*: pro $X \geq 0$ a $a > 0$ je $P(X \geq a) \leq \frac{\mathbb{E}X}{a}$. *Čebyševova*: $P(|X - \mathbb{E}X| \geq k) \leq \frac{\text{var } X}{k^2}$. Příklad: má-li $X \geq 0$ střední hodnotu 2, pak $P(X \geq 10) \leq 2/10 = 0,2$.

(c) *Zákon velkých čísel* (slabý): pro nezávislé stejně rozdělené se střední hodnotou μ platí $\bar{X}_n \xrightarrow{P} \mu$, tj. $\forall \varepsilon > 0 : P(|\bar{X}_n - \mu| > \varepsilon) \rightarrow 0$ (silný: konvergence skoro jistě). Binomické $\rightarrow$ Poissonovo pro velké n a malé p s $np = \lambda$ (zákon vzácných jevu); binomické $\rightarrow$ normální pro velké n (de Moivre-Laplace, speciální případ CLT) s $\mu = np$, $\sigma^2 = np(1-p)$.

$\text{Bi}(n, p)$: $\mathbb{E} = np$; geom.: $1/p$; $\text{Po}(\lambda)$: λ ; $\mathcal{N}(\mu, \sigma^2)$: μ ; $\text{Exp}(\lambda)$: $1/\lambda$. Markov $P(X \geq a) \leq \mathbb{E}X/a$; Čebyšev $P(|X - \mu| \geq k) \leq \sigma^2/k^2$. ZVČ: prumer $\rightarrow$ střední hodnota.

Kalibrace: Práce s konkrétními rozděleními a nerovnostmi je v požadavku pravděpodobnosti; úroveň 3 je shrnuje pro úplnost a podporuje statistické testy ze specializace.

Pozor (častá chyba): Geometrické: $\mathbb{E} = 1/p$, ne p . Markov potřebuje $X \geq 0$. Binomické $\rightarrow$ Poisson při malém p , $\rightarrow$ normální při velkém n .

Úloha 6. společná matematika [úroveň 3] Bodové a intervalové odhady

- (a) **1 b** Definujte bodový odhad, nestrannost a konzistenci odhadu.
- (b) **2 b** Popište metodu maximální věrohodnosti (MLE) a odvoďte MLE parametru p z n Bernoulliho pokusu s k úspěchy.
- (c) **3 b** Popište intervalový odhad střední hodnoty založený na normální aproximaci.

(a) *Bodový odhad* $\hat{\theta} = \hat{\theta}(X_1, \dots, X_n)$ je statistika odhadující neznámý parametr θ . Je *nestranný*, je-li $\mathbb{E}[\hat{\theta}] = \theta$ (žádné systematické vychýlení). Je *konzistentní*, pokud $\hat{\theta} \xrightarrow{P} \theta$ s rostoucím n (např. průměr $\bar{X}$ je nestranný a konzistentní odhad $\mathbb{E}X$).

(b) *MLE*: zvol θ maximalizující věrohodnost $L(\theta) = \prod_i P(x_i | \theta)$ (typicky maximalizujeme $\ln L$). Pro n Bernoulliho pokusu s k úspěchy: $L(p) = p^k(1-p)^{n-k}$, $\ln L = k \ln p + (n-k) \ln(1-p)$; derivace $\frac{k}{p} - \frac{n-k}{1-p} = 0$ dává $\hat{p} = \frac{k}{n}$ (relativní četnost); hraničně $k=0 \Rightarrow \hat{p}=0$ a $k=n \Rightarrow \hat{p}=1$.

(c) *Intervalový odhad* střední hodnoty: pro velké n je $\bar{X} \approx \mathcal{N}(\mu, \sigma^2/n)$ (CLT), takže $100(1-\alpha)\%$ konfidenční interval je

$$\bar{X} \pm z_{1-\alpha/2} \frac{\sigma}{\sqrt{n}}$$

($z_{1-\alpha/2}$ je kvantil normálního rozdělení, např. 1,96 pro 95 %); neznámé σ se nahradí výběrovou směrodatnou odchylkou s (pro malé n se použije t -kvantil).

Nestranný: $\mathbb{E}[\hat{\theta}] = \theta$. MLE: maximalizuj $\ln L(\theta) = \sum \ln P(x_i | \theta)$; pro Bernoulli $\hat{p} = k/n$. CI střední hodnoty: $\bar{X} \pm z_{1-\alpha/2} \sigma / \sqrt{n}$.

Kalibrace: Bodové i intervalové odhady jsou explicitně v požadavku statistiky a *codex* je označil za vysokovýnosové pro úplnost; podporují i ML (maximální věrohodnost).

Pozor (častá chyba): Nestrannost je o $\mathbb{E}[\hat{\theta}] = \theta$, ne o rozptylu. Šířka konfidenčního intervalu klesá jako $1/\sqrt{n}$ (čtyřnásobek dat = poloviční interval).

2 Společná informatika (rozšíření)

Úloha 7. **společná informatika** [úroveň 3] **Nativní vs interpretovaný běh, JIT/AOT, sestavení programu**

- (a) **1 b** Vysvětlíte rozdíl nativní vs interpretovaný běh, co je bytecode a interpret.
- (b) **2 b** Vysvětlíte JIT a AOT překlad a proces sestavení (oddělený překlad, linkování, staticky vs dynamicky linkované knihovny).
- (c) **3 b** Stručně: běhové prostředí procesu, volací konvence a relokační (position-independent code).

(a) *Nativní* běh: program je přeložen do strojového kódu procesoru a CPU ho vykonává přímo (rychlý, závislý na platformě). *Interpretovaný* běh: *interpret* čte a vykonává program (zdroj nebo *bytecode* = přenositelný mezikód virtuálního stroje, např. CIL/JVM) instrukci po instrukci - přenositelné, ale pomalejší.

(b) *JIT* (just-in-time): bytecode se překládá do nativního kódu *za běhu* těsně před prvním spuštěním metody (kombinuje přenositelnost s rychlostí, může optimalizovat dle běhových dat). *AOT* (ahead-of-time): překlad do nativního kódu *před* během (rychlý start, bez JIT režie). *Sestavení*: zdroje se *odděleně* přeloží na objektové soubory a *linker* je spojí (vyřeší odkazy mezi moduly) do spustitelného souboru. *Staticky* linkované knihovny se vkopírují do binárky (větší, soběstačná); *dynamické* (.dll/.so) se zavádějí *za běhu* (sdílené, menší binárka, závislost na verzi).

(c) *Běhové prostředí procesu*: adresový prostor (kód, data, halda, zásobník), vazba na OS přes systémová volání. *Volací konvence* určuje, jak se předávají argumenty (registry/zásobník), kdo

uklízí zásobník a kde je návratová hodnota. *Relokace* upravuje adresy při zavedení modulu na jinou adresu; *position-independent code* (PIC) používá relativní adresování, takže kód lze zavést na různé adresy s žádnými/menšími relokacemi v sekci kódu (dynamické relokace symbolu přes GOT/PLT mohou zustat); typické pro sdílené knihovny.

Nativní = přímo CPU; interpret = vykonává bytecode. JIT = překlad za běhu, AOT = předem. Sestavení: oddělený překlad → objektové soubory → linker. Statické knihovny v binárce, dynamické (.dll/.so) zaváděné za běhu.

Kalibrace: Řízení překladu a sestavení (JIT/AOT, linkování) je v požadavku programovacích jazyků, ale v moderních zkouškách vzácné - úroveň 3 to pokrývá pro úplnost.

Pozor (častá chyba): JIT překládá *za běhu*, ne před ním (to je AOT). Dynamické knihovny šetří paměť sdílením, ale přinášejí závislost na verzi („DLL hell”).

Úloha 8. **společná informatika** [úroveň 3] **Správa zdrojů a ošetření chyb (C#)**

- (a) **1 b** Co je správa životního cyklu zdroje (soubor, zámek, spojení) a jak souvisí s výjimkami?
- (b) **2 b** Vysvětlete `IDisposable` a `using` v C# (a obdobu RAII / try-with-resources). Co je deterministické uvolnění?
- (c) **3 b** Vysvětlete reference counting a finalizéry: kdy se volají a jaká jsou jejich úskalí.

(a) Zdroj (soubor, síťové spojení, zámek) je nutné po použití *uvolnit*, i když nastane chyba. Výjimka může běžný tok přerušit, takže uvolnění nesmí být jen za „šťastnou” cestou - musí proběhnout i při propagaci výjimky (jinak hrozí únik zdroje / zaseknutý zámek).

(b) V C# typ vlastníci zdroj implementuje `IDisposable` s metodou `Dispose()`. Konstrukce `using` zaručí volání `Dispose()` při opuštění bloku (i přes výjimku), což je *deterministické* uvolnění (přesně v daném bodě), na rozdíl od nedeterministického garbage collectoru.

```
using (var f = File.OpenRead(path)) {  
    // práce se souborem  
} // zde se volá f.Dispose() i když vyletela výjimka
```

Obdoby: RAII v C++ (destruktor uvolní zdroj při opuštění rozsahu), *try-with-resources* v Javě; bez nich slouží `try/finally`.

(c) *Reference counting* drží u objektu počet odkazu; při poklesu na 0 se objekt uvolní (deterministicky), ale neumí uvolnit *cykly*. Finalizér (~Trida / `Finalize`) volá GC *nedeterministicky* před uvolněním objektu jako záchrana pro nespravované zdroje - není zaručeno kdy (ani zda) se zavolá, proto se na něj nespolehá; správně je `Dispose()` (vzor `dispose` + finalizér jako pojistka).

Zdroj uvolní i při výjimce: C# `using+IDisposable` (deterministicky), C++ RAII (destruktor), jinak `try/finally`. Reference counting neuvolní cykly; finalizér běží nedeterministicky přes GC - nespolehat, použít `Dispose`.

Kalibrace: Správa zdroje a výjimky jsou v požadavku programovacích jazyků; jazyk je C# (volba majitele). Úroveň 3 doplňuje tuto tail oblast.

Pozor (častá chyba): Finalizér $\neq$ destruktor v C++ - běží nedeterministicky a není zaručen. Deterministické uvolnění zajistí `using/Dispose`, ne GC.

Úloha 9. **společná informatika** [úroveň 3] **Generické typy a funkcionální prvky (C#)**

- (a) **1 b** Co jsou generické typy a proč se používají (typová bezpečnost, výkon)?
- (b) **2 b** Co jsou typy reprezentující funkce: delegáti, `Func/Action`, lambda funkce?
- (c) **3 b** Napište generickou metodu v C# a vysvětlete rozdíl mezi generikami (C#) a šablonami (C++).

(a) *Generické typy* parametrizují třídu/metodu typem (`List<T>`, `Dictionary<K,V>`), takže lze psát kód nezávislý na konkrétním typu *bez ztráty typové kontroly* (vše se ověří při překladu) a *bez boxingu* hodnotových typu (na rozdíl od kontejneru `object`). Výsledek: méně duplicity, bezpečnější a rychlejší kód.

(b) *Delegát* je typ reprezentující odkaz na metodu (typový ukazatel na funkci). `Func<...,TResult>` a `Action<...>` jsou předdefinované generické delegáty (s/bez návratové hodnoty). *Lambda* je stručný zápis anonymní funkce, např. `x => x*x`, který lze přiřadit do delegátu a předat jako argument (funkce vyššího řádu).

(c)

```
T Max<T>(T a, T b) where T : IComparable<T>
    => a.CompareTo(b) >= 0 ? a : b;
// použití: Max(3, 7) -> 7; Max("a", "b") -> "b"
```

Generika (C#) jsou *reifikovaná* (typové parametry existují v metadatech/IL i za běhu, lze je omezit klauzulí `where`); JIT typicky sdílí kód pro referenční typy a specializuje pro hodnotové. *Šablony (C++)* jsou *instanciovány při překladu* - pro každý použitý typ vznikne samostatný kód (mocnější/statický polymorfismus, ale delší překlad a horší chybové hlášky).

Generika = parametrizace typem, typová bezpečnost bez boxingu. Delegát = typ funkce; `Func/Action`; lambda `x=>....` C# generika: jeden typ + běhová podpora + `where`; C++ šablony: instanciaci při překladu.

Kalibrace: *Generické typy a funkcionální prvky jsou v požadavku programovacích jazyků; úroveň 3 doplňuje tuto oblast v C#.*

Pozor (častá chyba): Generika C# nejsou „smazání typu“ jako v Javě - typové parametry jsou dostupné i za běhu. Lambda je výraz, delegát je typ, který ji drží.

Úloha 10. **společná informatika** [úroveň 3] **Garbage collection**

- (a) **1 b** Co je garbage collection a princip dosažitelnosti (kořeny, živé objekty)? Popište mark-and-sweep.
- (b) **2 b** Co je generační hypotéza a jak ji využívá generační GC?
- (c) **3 b** Porovnejte GC s reference countingem (problém cyklu) a vysvětlete stop-the-world a kompakci.

(a) *Garbage collector* automaticky uvolňuje objekty, které už nejsou potřeba. *Dosažitelnost*: objekt je *živý*, je-li dosažitelný z *kořenu* (lokální proměnné, registry, statická pole) přes řetěz referencí; nedosažitelné objekty jsou odpad. *Mark-and-sweep*: (1) *mark* - projdi z kořenu a označ všechny dosažitelné objekty; (2) *sweep* - uvolni neoznačené.

(b) *Generační hypotéza*: většina objektu umírá mladá (krátká životnost). *Generační GC* proto dělí haldu na generace; *mladou* generaci sbírá často a rychle (málo přeživších se povýší do starší), *starou* jen občas. Tím se zlevní typický průběh (nesbírá se pokaždé celá halda).

(c) *Reference counting* uvolňuje deterministicky při poklesu počtu odkazu na 0, ale *neuvolní*

cykly (vzájemně se odkazující objekty mají počet > 0 , ač jsou nedosažitelné) - sledovací GC (mark-sweep) cykly zvládá, protože jde od kořenu. *Stop-the-world*: po dobu (části) sběru se pozastaví běh aplikace, aby se graf objektu neměnil. *Kompakce*: po sweepu se živé objekty posunou k sobě, čímž se odstraní fragmentace haldy a zrychlí alokace.

GC uvolní nedosažitelné objekty (od kořenu). Mark-and-sweep: označ dosažitelné, uklid zbytek. Generační GC těží z „objekty umírají mladé“. Reference counting neuvolní cykly; GC ano. Stop-the-world pozastaví aplikaci, kompakce odstraní fragmentaci.

Kalibrace: *Garbage collection je v požadavku (správa paměti) a objevila se v dřívějších zkouškách; úroveň 3 ji pokrývá do hloubky.*

Pozor (častá chyba): Reference counting selhává na cyklech - to je klíčový rozdíl proti sledovacímu GC. Generační GC nesbírá pokaždé celou haldu.

3 Specializace: Strojové učení (rozšíření)

Úloha 11. **specializace UI-SU [úroveň 3]** **Klasifikace do více tříd (softmax) a křížová validace**

- (a) **1 b** Definujte softmax pro klasifikaci do K tříd a uveďte alternativu one-vs-rest.
- (b) **2 b** Napište ztrátovou funkci (kategoriální křížová entropie) a gradient pro jeden krok.
- (c) **3 b** Popište k -násobnou křížovou validaci a spočítejte průměrnou přesnost z foldu $\{0,80; 0,85; 0,75; 0,90; 0,80\}$.

(a) *Softmax* převede skóre $z_k = w_k^\top x$ na pravděpodobnosti tříd:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad \sum_k p_k = 1.$$

Predikuje se třída s nejvyšším p_k . *One-vs-rest*: natrénuj K binárních klasifikátorů (každý „třída k vs zbytek“) a vyber ten s nejvyšším skóre - jednodušší, ale skóre nejsou kalibrovaná pravděpodobnost.

(b) *Kategoriální křížová entropie* pro správnou třídu y : $L = -\ln p_y = -\ln \frac{e^{z_y}}{\sum_j e^{z_j}}$. Gradient vůči skóre má stejně elegantní tvar jako u logistické regrese:

$$\frac{\partial L}{\partial z_k} = p_k - \mathbb{I}[k=y], \quad \nabla_{w_k} L = (p_k - \mathbb{I}[k=y])x.$$

Krok SGD: $w_k \leftarrow w_k - \eta(p_k - \mathbb{I}[k=y])x$.

(c) *k-fold CV*: rozděl data na k částí, k -krát trénuj na $k-1$ z nich a vyhodnoť na zbylé; výsledky zprůměruj (robustnější odhad než jeden split, využije všechna data). Použije se i k volbě hyperparametru (vyber hodnotu s nejlepší průměrnou CV přesností). Průměr: $\frac{0,80+0,85+0,75+0,90+0,80}{5} = \frac{4,10}{5} = 0,82$.

Softmax $p_k = e^{z_k} / \sum_j e^{z_j}$; ztráta $-\ln p_y$; gradient $(p_k - \mathbb{I}[k=y])x$. *k-fold CV*: průměr přesností přes foldy; vyber hyperparametr s nejlepším průměrem.

Kalibrace: Klasifikace do více tříd (softmax) je v požadavku logistické regrese; CV je opakované evaluační téma. Úroveň 3 doplňuje multiclass variantu.

Pozor (častá chyba): Softmax normuje přes všechny třídy (jmenovatel je suma). CV se průměruje přes foldy; na testu se nevybírám hyperparametr (jen na dev/CV).

4 Specializace: Základy UI (rozšíření)

Úloha 12. specializace UI-SU [úroveň 3] POMDP a aktualizace beliefu

Částečně pozorovatelný MDP se dvěma stavy s_1, s_2 . Pro akci a : $T(s_1 \rightarrow s_1) = 0,8$, $T(s_1 \rightarrow s_2) = 0,2$, $T(s_2 \rightarrow s_1) = 0,3$, $T(s_2 \rightarrow s_2) = 0,7$. Model pozorování: $O(o | s_1) = 0,9$, $O(o | s_2) = 0,2$. Počáteční belief $b = (0,5, 0,5)$.

- (a) **1 b** Definujte POMDP a pojem belief (víra o stavu).
- (b) **2 b** Napište vzorec aktualizace beliefu po akci a a pozorování o .
- (c) **3 b** Spočítejte nový belief po provedení a a pozorování o .

(a) $POMDP = (S, A, T, R, \Omega, O, \gamma)$: jako MDP, ale agent stav přímo *nevidí*; po akci dostane jen pozorování $o \in \Omega$ s pravděpodobnostmi $O(o | s', a)$ (obecně $O : \Omega \times S \times A \rightarrow [0, 1]$ - pozorování smí záviset i na akci). Belief b je rozdělení pravděpodobnosti nad stavy (co agent ví); rozhoduje se podle něj (POMDP lze chápat jako MDP nad spojitým prostorem beliefu).

(b) Aktualizace (předpověď přes přechod + korekce pozorováním, Bayes):

$$b'(s') \propto O(o | s', a) \sum_s T(s' | s, a) b(s),$$

poté normalizace, aby $\sum_{s'} b'(s') = 1$.

(c) (V tomto příkladu O na akci nezávisí, tj. $O(o | s')$.) Predikce: $\tilde{b}(s_1) = 0,8 \cdot 0,5 + 0,3 \cdot 0,5 = 0,55$, $\tilde{b}(s_2) = 0,2 \cdot 0,5 + 0,7 \cdot 0,5 = 0,45$. Korekce pozorováním o : $\propto (0,9 \cdot 0,55, 0,2 \cdot 0,45) = (0,495, 0,09)$. Normalizace ($/0,585$): $b'(s_1) \approx 0,846$, $b'(s_2) \approx 0,154$.

POMDP = MDP + skrytý stav + pozorování $O(o | s', a)$; agent drží belief b (rozdělení nad stavy). Update: $b'(s') \propto O(o | s') \sum_s T(s' | s, a) b(s)$, pak normuj. (Stejná logika jako filtrace HMM s akcemi.)

Kalibrace: POMDP (základní definice) je explicitně v požadavku Základu UI. Belief update je stejný mechanismus jako filtrace HMM - úroveň 3 ho doplňuje numericky.

Pozor (častá chyba): Belief update = predikce (přechod) PAK korekce (pozorování) + normalizace, jako u HMM. Belief je rozdělení, ne jeden „nejpravděpodobnější“ stav.

Úloha 13. specializace UI-SU [úroveň 3] Rezoluce v predikátové logice a unifikace

- (a) **1 b** Co je unifikace a nejobecnější unifikátor (MGU)? Co je skolemizace?
- (b) **2 b** Z předpokladu „ $\forall x (P(x) \rightarrow Q(x))$ “ a „ $P(a)$ “ rezolucí odvoďte $Q(a)$.
- (c) **3 b** Vysvětlete, proč je rezoluce důkaz sporem a jakou roli hraje převod do klauzulární formy.

(a) *Unifikace* hledá substituci proměnných, která dva literály (termy) ztotožní; *nejobecnější unifikátor* (MGU) je ta nejméně specializující (každá jiná unifikace je její instancí). *Skolemizace* odstraní existenční kvantifikátory při převodu do klauzulární formy: $\exists y$ se nahradí novou *Skolemovou* konstantou (není-li pod $\forall$) nebo *Skolemovou funkcí* závislých univerzálních proměnných. Skolemizace zachovává *splnitelnost* (equisatisfiability), ne obecně logickou ekvivalenci - rezoluci to stačí, protože dokazuje sporem (jde jí o nespjitelnost).

(b) Klauzulární forma: $\forall x(P(x) \rightarrow Q(x))$ dá klauzuli $\{\neg P(x), Q(x)\}$; dále $\{P(a)\}$. Pro důkaz $Q(a)$ přidáme negaci cíle $\{\neg Q(a)\}$.

- Rezolvuj $\{\neg P(x), Q(x)\}$ a $\{P(a)\}$ s MGU $\{x/a\}$: vznikne $\{Q(a)\}$.
- Rezolvuj $\{Q(a)\}$ a $\{\neg Q(a)\}$: vznikne prázdná klauzule $\square$.

Odvození prázdné klauzule dokazuje $Q(a)$.

(c) Rezoluce dokazuje $T \models \varphi$ *sporem*: k teorii přidá $\neg\varphi$ a snaží se odvodit *prázdnou klauzuli* (= nespjitelnost $T \cup \{\neg\varphi\}$). Aby to šlo mechanicky, převede se vše do *klauzulární formy* (PNF $\rightarrow$ skolemizace $\rightarrow$ CNF $\rightarrow$ množina klauzulí); rezoluce s unifikací je pak úplná zamítací procedura pro predikátovou logiku.

Unifikace = substituce ztotožňující literály, MGU = nejobecnější. Skolemizace odstraní $\exists$. Rezoluce: přidej $\neg$ cíl, převed na klauzule, rezolvuj s MGU do prázdné klauzule (důkaz sporem).

Kalibrace: Rezoluce a Hornovy klauzule jsou v požadavku Základu UI i logiky; úroveň 3 doplňuje predikátovou rezoluci s unifikací (těžší než výroková).

Pozor (častá chyba): Skolemizace ruší $\exists$, ne $\forall$. Rezoluce dokazuje sporem - bez přidání negace cíle nic nedokážeš.

Úloha 14. specializace UI-SU [úroveň 3] Plánování zpětnou regresí (STRIPS)

Operátor **Stack(A,B)**: předpoklady $\{Hold(A), Clear(B)\}$, přidává $\{On(A,B), Clear(A), HandEmpty\}$, ruší $\{Hold(A), Clear(B)\}$. Cíl je $On(A,B)$.

- 1 b** Připomeňte princip zpětného (regresního) plánování.
- 2 b** Proveďte jeden krok regrese cíle $On(A,B)$ přes operátor **Stack(A,B)**.
- 3 b** Porovnejte dopředné a zpětné plánování (větvení, relevance operátoru).

(a) *Zpětné (regresní) plánování* jde od cíle: vyber operátor, jehož *přidávací* efekty obsahují nějaký cílový literál, a *regreduj* cíl - nahraď splněné cílové literály předpoklady operátoru. Pracuje jen s *relevantními* operátory (těmi, co k cíli přispívají).

(b) Cíl $G = \{On(A,B)\}$. Operátor **Stack(A,B)** přidává $On(A,B)$, je tedy relevantní a *konzistentní* (neruší žádný jiný cílový literál). Regrese:

$$G' = (G \setminus \text{add}(\text{Stack})) \cup \text{pre}(\text{Stack}) = (\{On(A,B)\} \setminus \{On(A,B), \dots\}) \cup \{Hold(A), Clear(B)\}.$$

Nový podcíl $G' = \{Hold(A), Clear(B)\}$ - tj. abychom dosáhli $On(A,B)$ pomocí **Stack(A,B)**, musíme nejprve dosáhnout, že držíme A a B je volné.

(c) *Dopředné* plánování jde od počátečního stavu a aplikuje použitelné operátory; má velký větvící faktor (mnoho použitelných akcí), ale konkrétní úplné stavy. *Zpětné* jde od cíle a uvažuje jen *relevantní* operátory (menší větvení k cíli), ale pracuje s *částečnými* stavy (množinami podcílů) a musí hlídat konzistenci (operátor nesmí rušit jiný cílový literál).

Jádro na 2 body (obě mají napájení = jádro > 2)

Regrese cíle G přes operátor o (jehož add pokrývá část G a nic z G neruší): $G' = (G \setminus \text{add}(o)) \cup \text{pre}(o)$. Zpětné plánování = relevantní operátory od cíle; dopředné = použitelné operátory od počátku.

Kalibrace: Automatické plánování (dopředné/zpětné) je v požadavku Základu UI; úroveň 3 přidává vypracovaný regresní krok ke konceptu z úrovně 1.

Pozor (častá chyba): Regrese odečítá z cíle add efekty a přidává *předpoklady*; operátor je použitelný jen, neruší-li jiný cílový literál (konzistence).

5 Zkoušky nanečisto a dril

Zkouška nanečisto 1

Pokyny

8 otázek (2 matematika, 2 informatika, 4 specializace), každá 0-3 body. Cíl: společné $\geq 5/12$ a specializace $\geq 7/12$. Zakryj řešení, vyřeš na papír, pak porovnej.

Otázka 1 matematika Průběh funkce

Najděte lokální extrémy funkce $f(x) = x^3 - 3x$ a spočítejte $\lim_{x \rightarrow 0} \frac{1 - \cos x}{x^2}$.

$f'(x) = 3x^2 - 3 = 0 \Rightarrow x = \pm 1$; $f'' = 6x$, takže $x = -1$ je maximum ($f = 2$), $x = 1$ minimum ($f = -2$). Limita: $1 - \cos x = \frac{x^2}{2} + o(x^2)$, tedy $\lim = \frac{1}{2}$.

Otázka 2 matematika Vlastní čísla a diagonalizace

Pro $A = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$ najděte vlastní čísla, vlastní vektory a rozhodněte o diagonalizovatelnosti.

$\det(A - \lambda I) = (1 - \lambda)^2 - 4 = \lambda^2 - 2\lambda - 3 = (\lambda - 3)(\lambda + 1)$, takže $\lambda = 3, -1$. Vektory: $\lambda = 3 \rightarrow (1, 1)$, $\lambda = -1 \rightarrow (1, -1)$. Dvě různá vlastní čísla $\Rightarrow$ diagonalizovatelná, $A = PDP^{-1}$, $P = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$, $D = \text{diag}(3, -1)$.

Otázka 3 informatika Konečný automat

Sestrojte DFA pro binární čísla (čtená odshora) dělitelná 3.

Stavy = zbytek po dělení 3: $\{0, 1, 2\}$, počátek a jediný přijímající stav 0. Přejít při bitu b : $q \rightarrow (2q + b) \bmod 3$. Tedy $0 \xrightarrow{0} 0$, $0 \xrightarrow{1} 1$, $1 \xrightarrow{0} 2$, $1 \xrightarrow{1} 0$, $2 \xrightarrow{0} 1$, $2 \xrightarrow{1} 2$. (Funguje, protože přečtení bitu odpovídá $n \mapsto 2n + b$.)

Otázka 4 informatika Virtuální paměť

Stránka 4 KiB. Přeložte virtuální adresu 0x5678, je-li stránka 5 namapována na rámec 2. Co je race condition?

Offset 12 bitu: offset = 0x678, číslo stránky = 0x5. Stránka 5 → rámec 2, fyzická adresa = $(2 \ll 12) \mid 0x678 = 0x2678$. *Race condition*: výsledek závisí na prokládání vláken nad sdíleným stavem; řeší se vzájemným vyloučením (zámek/kritická sekce).

Otázka 5 specializace Logistická regrese

Napište predikci, ztrátu a jeden krok SGD pro $w = (0, 0)$, $x = (1, 1)$, $y = 1$, $\eta = 0,5$.

$p = \sigma(0) = 0,5$; ztráta NLL = $-\ln p$. Gradient $(p - y)x = (-0,5, -0,5)$, krok $w \leftarrow (0, 0) - 0,5 \cdot (-0,5, -0,5) = (0,25, 0,25)$. Nová predikce $\sigma(0,5) \approx 0,62$ (posun k 1).

Otázka 6 specializace Rozhodovací strom

Uzel se 4 kladnými a 4 zápornými příklady. Spočítejte entropii a informační zisk štěpení na čisté listy $(4, 0)$ a $(0, 4)$.

$H = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1$ bit. Čisté listy mají $H = 0$, vážená entropie 0, takže $IG = 1 - 0 = 1$ bit (dokonalé rozdělení).

Otázka 7 specializace Shlukování k-means

Body 1, 2, 8, 9 na přímce, $k = 2$, počáteční centra 1 a 9. Provedte jednu iteraci.

Přiřazení: $1, 2 \rightarrow c_1=1$ (blíží), $8, 9 \rightarrow c_2=9$. Nová centra: $c_1 = \frac{1+2}{2} = 1,5$, $c_2 = \frac{8+9}{2} = 8,5$. Další iterace už nic nezmění (konvergence), ztráta $J = 0,25 \cdot 4 = 1,0$.

Otázka 8 specializace Bayesovská inference

$P(A) = 0,3$, $P(B | A) = 0,8$, $P(B | \neg A) = 0,4$. Spočítejte $P(A | B)$.

$P(A, B) = 0,3 \cdot 0,8 = 0,24$, $P(\neg A, B) = 0,7 \cdot 0,4 = 0,28$, $P(B) = 0,52$. $P(A | B) = \frac{0,24}{0,52} \approx 0,46$.

Bodování (orientačně)

Každá úplná odpověď ≈ 3 , definice + hlavní postup ≈ 2 . Na průchod stačí: u 4 společných (Q1-Q4) nasbírat ≥ 5 (např. 2+2+1+1), u 4 specializačních (Q5-Q8) ≥ 7 (např. 2+2+2+1). Pokud někde vyjde ≤ 1 , dožen to jinde - hlídej obě podlahy zvlášť.

Zkouška nanečisto 2

Pokyny

8 otázek (2 matematika, 2 informatika, 4 specializace), 0-3 body. Cíl: společné $\geq 5/12$ a specializace $\geq 7/12$.

Otázka 1 matematika Integrály

Spočtete $\int_0^2 (x^2 - 1) dx$ a obsah mezi $y = x$ a $y = x^2$ na $[0, 1]$.

$$\int_0^2 (x^2 - 1) dx = \left[\frac{x^3}{3} - x\right]_0^2 = \frac{8}{3} - 2 = \frac{2}{3}. \text{ Obsah} = \int_0^1 (x - x^2) dx = \left[\frac{x^2}{2} - \frac{x^3}{3}\right]_0^1 = \frac{1}{2} - \frac{1}{3} = \frac{1}{6}.$$

Otázka 2 **matematika** Teorie grafu

Definujte barevnost. Je K_5 rovinný? Zdůvodněte přes $E \leq 3V - 6$.

Barevnost $\chi(G)$ = nejmenší počet barev pro dobré obarvení (sousedé různě). K_5 : $V = 5$, $E = \binom{5}{2} = 10$; rovinný jednoduchý graf splňuje $E \leq 3V - 6 = 9$. Protože $10 > 9$, K_5 rovinný *není*. ($\chi(K_5) = 5$.)

Otázka 3 **informatika** Složitost

Definujte třídy P a NP a NP-úplnost. Jak se dokazuje NP-úplnost?

P = řešitelné v polynomiálním čase; NP = ano-instance mají polynomiálně ověřitelný certifikát. B je NP-úplný, je-li v NP a NP-těžký ($\forall K \in \text{NP}: K \leq_p B$). Důkaz: (1) ukázat $B \in \text{NP}$, (2) polynomiální redukce ze *známého* NP-úplného problému na B.

Otázka 4 **informatika** Reprezentace dat

Jak je v little-endian uloženo `int32 0x01020304`? Co je zarovnání?

Bajty od nejnižší adresy: 04 03 02 01 (nejméně významný první). Zarovnání: hodnota velikosti *s* začíná na adrese dělitelné *s* (rychlejší přístup CPU); překladač vkládá výplň mezi položky struktury.

Otázka 5 **specializace** Lineární regrese a L2

Napište normální rovnice OLS a ridge. Jaký má L2 efekt na koeficienty?

OLS: $X^T X w = X^T y$, $w = (X^T X)^{-1} X^T y$ (je-li $X^T X$ regulární; jinak pseudoinverze). Ridge: $(X^T X + \lambda I) w = X^T y$ (bias se obvykle nepenalizuje). L2 koeficienty *smršťuje* k nule (menší rozptyl, řešitelné i při kolinearitě).

Otázka 6 **specializace** Evaluace klasifikátoru

Z 1000 vzorku je 5 % pozitivních; recall 0,8, precision 0,5. Spočtete TP, FP, FN, TN a F_1 .

$P = 50, N = 950$. $TP = 0,8 \cdot 50 = 40$, $FN = 10$; z precision $TP + FP = 80 \Rightarrow FP = 40$, $TN = 910$. $F_1 = \frac{2 \cdot 0,5 \cdot 0,8}{1,3} \approx 0,615$. (Accuracy 0,95 je zde zavádějící - nevyvážené třídy.)

Otázka 7 **specializace** SVM a PCA

Vysvětlete maximální margin a podpůrné vektory. Co PCA optimalizuje a jak souvisí s kovariancí?

SVM hledá nadrovinu s největším marginem ($\min \frac{1}{2} \|w\|^2$ s $y_i(w^\top x_i + b) \geq 1$); určují ji nejbližší body = podpůrné vektory. PCA hledá směry maximálního rozptylu = vlastní vektory kovarianční matice (top- k podle vlastních čísel); ekvivalentně minimalizuje rekonstrukční chybu.

Otázka 8 specializace MDP a Bellman

Napište Bellmanovu rovnici pro V^* a popište iteraci hodnot.

$V^*(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^*(s')]$. Iterace hodnot: inicializuj V , opakovaně aplikuj pravou stranu jako update pro všechny stavy do konvergence; pak $\pi^*(s) = \arg \max_a (\dots)$.

Bodování (orientačně)

Společné Q1-Q4: cíl ≥ 5 . Specializace Q5-Q8: cíl ≥ 7 (tři ze čtyř na ≥ 2). Hlídej obě podlahy nezávisle.

Tvrďý specializační drill

Pokyny

6 specializačních otázek za sebou (numerické). Vázající podlaha je specializace ($\geq 7/12$, tři ze čtyř na ≥ 2). Trénuj rychlost a přesnost výpočtu.

Otázka 1 specializace EM: responsibility

$\pi = (0,5, 0,5)$, $p(x=1 | 1) = 0,7$, $p(x=1 | 2) = 0,3$. Spočítejte responsibility bodu $x=1$.

$$r_1 = \frac{0,5 \cdot 0,7}{0,5 \cdot 0,7 + 0,5 \cdot 0,3} = \frac{0,35}{0,5} = 0,7, r_2 = 0,3.$$

Otázka 2 specializace Matice záměn

$TP=30, FP=10, FN=20, TN=40$. Spočítejte precision, recall, F_1 a accuracy.

$$\text{precision} = \frac{30}{40} = 0,75, \text{recall} = \frac{30}{50} = 0,6, F_1 = \frac{2 \cdot 0,75 \cdot 0,6}{1,35} \approx 0,667, \text{accuracy} = \frac{70}{100} = 0,7.$$

Otázka 3 specializace HMM: filtrace

Prior $(0,6, 0,4)$; přechod $P(t | t) = 0,7, P(t | f) = 0,2$; emise $P(o | t) = 0,8, P(o | f) = 0,1$. Spočítejte $P(\cdot | o)$.

Predikce: $P(t) = 0,7 \cdot 0,6 + 0,2 \cdot 0,4 = 0,5, P(f) = 0,5$. Korekce: $\propto (0,8 \cdot 0,5, 0,1 \cdot 0,5) = (0,4, 0,05)$, norm $/0,45 \rightarrow (0,889, 0,111)$.

Otázka 4 specializace Bayes-optimal predikce

$h_1:P(+)=0,9$ (prior 0,4), $h_2:P(+)=0,5$ (prior 0,6). Po pozorování „+“ spočtěte posterior a $P(\text{další}=+)$.

Posterior $\propto (0,4 \cdot 0,9, 0,6 \cdot 0,5) = (0,36, 0,30)$, norm $/0,66 \rightarrow (0,545, 0,455)$. Predikce $= 0,9 \cdot 0,545 + 0,5 \cdot 0,455 \approx 0,718$.

Otázka 5 specializace Ensemble: majoritní hlasování

Tři nezávislé klasifikátory, každý přesnost 0,7, majoritní hlasování. Spočtěte přesnost a uveďte nejlepší/nejhorsí případ.

Při nezávislosti $P(\geq 2 \text{ správně}) = \binom{3}{2} 0,7^2 \cdot 0,3 + 0,7^3 = 3 \cdot 0,147 + 0,343 = 0,784$. Nejlepší případ (chyby se nepřekrývají) až 1; jsou-li chyby dokonale korelované, ensemble nepomůže a zůstává 0,7; při nejnepříznivější závislosti (maximální překryv dvojic chyb) může klesnout až k 0,55. Ensemble pomáhá při *nekorelovaných* chybách.

Otázka 6 specializace Iterace hodnot

Stavy A, B , $\gamma = 0,9$. Z A akce do B (odměna 0), z B akce do cíle (odměna 10). Spočtěte $V^*(A), V^*(B)$.

$V^*(B) = 10 + 0,9 \cdot 0 = 10$. $V^*(A) = 0 + 0,9 \cdot V^*(B) = 0,9 \cdot 10 = 9$. (Iterace: $V_1(B) = 10$, $V_2(A) = 9$.)

Vyhodnocení

Ber to jako dva specializační bloky po čtyřech. V každém čtyřbloku miř na ≥ 7 : tři otázky vyřešené úplně (3×2) plus jedna alespoň částečně (1) stačí. Pokud ti numerika padá pod 2 body, zopakuj příslušnou úlohu z úrovně 1-2.